

CHAPITRE V SUITE A

CLE PRIMAIRE ET ETRANGERE

I. INTRODUCTION

Maintenant que les index n'ont plus de secret pour vous, nous allons passer à une autre notion très importante : les clés.

Les clés sont, vous allez le voir, intimement liées aux index. Et tout comme **NOT NULL** et les index **UNIQUE**, les clés font partie de ce qu'on appelle les **contraintes (CONSTRAINT)**. Il existe deux types de clés :

- ❖ Les **clés primaires**, qui ont déjà été survolées lors des chapitres précédents sur la création des tables, et qui servent à **identifier une ligne** de manière unique;
- ❖ Les **clés étrangères**, qui permettent de gérer des **relations entre plusieurs tables**, et garantissent la **cohérence des données**.

II. CLÉS PRIMAIRES, LE RETOUR

Les clés primaires ont déjà été introduites dans le chapitre de création des tables. Je vous avais alors donné la définition suivante :

La clé primaire d'une table est une contrainte d'unicité, composée d'une ou plusieurs colonnes, et qui permet d'identifier de manière unique chaque ligne de la table.

Examinons plus en détail cette définition.

Contrainte d'unicité : ceci ressemble fort à un index **UNIQUE**.

Composée d'une ou plusieurs colonnes : comme les index, les clés peuvent donc être composites.

Permet d'identifier chaque ligne de manière unique : dans ce cas, une clé primaire ne peut pas être **NULL**.

Ces quelques considérations résument très bien l'essence des clés primaires. En gros, une clé primaire est un index **UNIQUE** sur une colonne qui ne peut pas être **NULL**.

D'ailleurs, vous savez déjà que l'on définit une clé primaire grâce aux mots-clés **PRIMARY KEY**. Or, nous avons vu dans le précédent chapitre que **KEY** s'utilise pour définir un index. Par conséquent, lorsque vous définissez une clé primaire, pas besoin de définir en plus un index sur la (les) colonne(s) qui compose(nt) celle-ci, c'est déjà fait ! Et pas besoin non plus de rajouter une contrainte **NOT NULL**.

Pour le dire différemment, une contrainte de clé primaire est donc une combinaison de deux des contraintes que nous avons vues jusqu'à présent : **UNIQUE** et **NOT NULL**.

III. CHOIX DE LA CLE PRIMAIRE

Le choix d'une clé primaire est une étape importante dans la conception d'une table. Ce n'est pas parce que vous avez l'impression qu'une colonne, ou un groupe de colonnes, pourrait faire une bonne clé primaire que c'est le cas. Reprenons l'exemple d'une table *Client*, qui contient le nom, le prénom, la date de naissance et l'email des clients d'une société.

Chaque client a bien sûr un nom et un prénom. Est-ce que (*nom, prénom*) ferait une bonne clé primaire ? Non bien sûr : il est évident ici que vous risquez des doublons.

Et si on ajoute la date de naissance ? Les chances de doublons sont alors quasi nulles. Mais quasi nul, ce n'est pas nul...

Qu'arrivera-t-il le jour où vous voyez débarquer un client qui a les mêmes nom et prénom qu'un autre, et qui est né le même jour ?

On refait toute la base de données ? Non, bien sûr.

Et l'email alors ? Il est impossible que deux personnes aient la même adresse email, donc la contrainte d'unicité est respectée. Par contre, tout le monde n'est pas obligé d'avoir une adresse email. Difficile donc de mettre une contrainte **NOT NULL** sur cette colonne.

Par conséquent, on est bien souvent obligé d'ajouter une colonne pour jouer le rôle de la clé primaire. C'est cette fameuse colonne *id*, auto-incrémentée que nous avons déjà vue pour la table *Animal*.

Il y a une autre raison d'utiliser une colonne spéciale auto-incrémentée, de type **INT** (ou un de ses dérivés) pour la clé primaire.

En effet, si l'on définit une clé primaire, c'est en partie dans le but d'utiliser au maximum cette clé pour faire des recherches dans la table. Bien sûr, parfois ce n'est pas possible, parfois vous ne connaissez pas l'*id* du client, et vous êtes obligés de faire une recherche par nom. Cependant, vous verrez bientôt que les clés primaires peuvent servir à faire des recherches de manière **indirecte** sur la table. Du coup, comme les recherches sont beaucoup plus rapides sur des nombres que sur des textes, il est souvent intéressant d'avoir une clé primaire composée de colonnes de type **INT**.

Enfin, il y a également l'argument de l'auto-incrémentation. Si vous devez remplir vous-mêmes la colonne de la clé primaire, étant donné que vous êtes humains (comment ça pas du tout ?), vous risquez de faire une erreur. Avec une clé primaire auto incrémentée, vous ne risquez rien : MySQL fait tout pour vous. De plus, on ne peut définir une colonne comme auto-incrémentée que si elle est de type **INT** et qu'il existe un index dessus. Dans le cas d'une clé primaire auto-incrémentée, on définit généralement la colonne comme un entier UNSIGNED, comme on l'a fait pour la table *Animal*.



Il peut bien sûr n'y avoir qu'une seule clé primaire par table. De même, une seule colonne peut être auto-incrémentée (la clé primaire en général).

PRIMARY KEY or not PRIMARY KEY

Je me dois de vous dire que d'un point de vue technique, avoir une clé primaire sur chaque table n'est pas obligatoire. Vous pourriez travailler toute votre vie sur une base de données sans aucune clé primaire, et ne jamais voir un message d'erreur à ce propos.

Cependant, d'un point de vue **conceptuel**, ce serait une grave erreur. Ce n'est pas le propos de ce tutoriel que de vous enseigner les étapes de conception d'une base de données mais, s'il vous plaît, pensez à mettre une clé primaire sur chacune de vos tables. Si l'utilité n'en est pas complètement évidente pour vous pour le moment, elle devrait le devenir au fur et à mesure de votre lecture.

IV. CREATION D'UNE CLE PRIMAIRE

Donc, à nouveau, la clé primaire peut être créée en même temps que la table, ou par la suite.

Lors de la création de la table

On peut donc préciser **PRIMARY KEY** dans la description de la colonne qui doit devenir la clé primaire (pas de clé composite dans ce cas) :

Exemple : création de la table *Animal* en donnant la clé primaire dans la description de la colonne.

Code : SQL

```
CREATE TABLE Animal (  
    id SMALLINT AUTO_INCREMENT PRIMARY KEY,  
    espece VARCHAR(40) NOT NULL,  
    sexe CHAR(1),  
    date_naissance DATETIME NOT NULL,  
    nom VARCHAR(30),  
    commentaires TEXT  
)  
ENGINE=InnoDB;
```

```
mysql>  
mysql> create table animal(id smallint auto_increment primary key,espece varchar(40),sexe char(2),ddn date,nom varchar(30),commentaires text) engine=innodb;  
Query OK, 0 rows affected (0.15 sec)
```

D'après la description :

```
mysql>
mysql> desc animal;
```

Field	Type	Null	Key	Default	Extra
id	smallint(6)	NO	PRI	NULL	auto_increment
espece	varchar(40)	YES		NULL	
sexe	char(2)	YES		NULL	
ddn	date	YES		NULL	
nom	varchar(30)	YES		NULL	
commentaires	text	YES		NULL	

```
6 rows in set (0.32 sec)
```

Ou bien, on ajoute la clé à la suite des colonnes.

Code : SQL

```
CREATE TABLE [IF NOT EXISTS] Nom_table (
    colonne1 description_colonne1 [,
    colonne2 description_colonne2,
    colonne3 description_colonne3,
    ...],
    [CONSTRAINT [symbole_contrainte]] PRIMARY KEY (colonne_pk1 [,
    colonne_pk2, ...]) -- comme pour les index UNIQUE, CONSTRAINT est
    facultatif
)
[ENGINE=moteur];
```

C'est ce que nous avons fait d'ailleurs pour la table *Animal*. Cette méthode permet bien sûr la création d'une clé composite (avec plusieurs colonnes).

Exemple : création d'*Animal*.

Code : SQL

```
CREATE TABLE Animal (
    id SMALLINT AUTO_INCREMENT,
    espece VARCHAR(40) NOT NULL,
    sexe CHAR(1),
    date naissance DATETIME NOT NULL,
    nom VARCHAR(30),
    commentaires TEXT,
    PRIMARY KEY (id)
)
ENGINE=InnoDB;
```

Après création de la table

On peut toujours utiliser **ALTER TABLE**. Par contre, **CREATE INDEX** n'est pas utilisable pour les clés primaires.



Si vous créez une clé primaire sur une table existante, assurez-vous que la (les) colonne(s) où vous souhaitez l'ajouter ne contienne(nt) pas **NULL**.

Code : SQL

```
ALTER TABLE nom_table  
ADD [CONSTRAINT [symbole_contrainte]] PRIMARY KEY (colonne_pk1 [,  
colonne_pk2, ...]);
```

APPLICATION

```
mysql> create table lo(id int, nom varchar(30));  
Query OK, 0 rows affected (0.34 sec)  
  
mysql> desc lo;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| id    | int(11)       | YES  |     | NULL    |       |  
| nom   | varchar(30)   | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> alter table lo add constraint symbole_d_la_cle primary key(id,nom);  
Query OK, 0 rows affected (0.12 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql>
```

V. SUPPRESSION DE LA CLE PRIMAIRE

Code : SQL

```
ALTER TABLE nom_table  
DROP PRIMARY KEY
```

APPLICATION

```
mysql>
mysql> alter table lo drop primary key;
Query OK, 0 rows affected (0.34 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

Pas besoin de préciser de quelle clé il s'agit, puisqu'il ne peut y en avoir qu'une seule par table !

VI. QUESTIONS PERTINENTES

1. Est-il possible d'ajouter une clé primaire à travers plusieurs colonnes au sein d'une table ?

Oui, effectivement. Nous l'avons évalué ci-dessus. Voici donc en bref les étapes requises pour le réaliser :

Soit la table ci-dessous contenant aucune clé:

```
mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   |     | 0        |       |
| nom    | varchar(30)   | NO   |     |          |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.12 sec)
```

Ajoutant maintenant une clé primaire à chacune de ses colonnes simultanément:

```
mysql> alter table lo add constraint key_symbole primary key (id,nom);
Query OK, 0 rows affected (0.23 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Après description :

```
mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | int(11)       | NO   | PRI | 0        |       |
| nom    | varchar(30)   | NO   | PRI |          |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Chose promise, chose faite.

NB : Sans pourtant extrapoler, le bon sens exige qu'il puisse y voir qu'une seule clé primaire par table, mais MySQL nous permet d'y insérer plusieurs par **contrainte (constraint)** comme nous l'avions fait ci-dessous.

2. Comment effacer simultanément toutes les clés primaires présentes dans notre table ?

Là encore, c'est une chose déjà très réalisable, car nous l'avions aussi eu à l'exécuter ci-dessus. Bref, ci-dessous est la syntaxe requise pour le faire :

Soit la table suivante contenant plusieurs clés primaires:

```
mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   | PRI |          |       |
| nom   | varchar(30)   | NO   | PRI |          |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Pour les effacer, il suffit juste de saisir la syntaxe ci-dessous :

```
mysql> alter table lo drop primary key;
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   |     |          |       |
| nom   | varchar(30)   | NO   |     |          |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.02 sec)
```

3. Comment le faire simultanément avec la clé unique ?

Chose très simple, il suffit juste de suivre le style de la requête ci-dessous :

Soit la table ci-dessous contenant aucune clé:

```
mysql> desc lo;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO		0	
nom	varchar(30)	NO			

```
2 rows in set (0.02 sec)
```

Après traitement, nous aurons ce qui suit:

```
mysql> alter table lo add unique (id,nom);
Query OK, 0 rows affected (0.34 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc lo;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	0	
nom	varchar(30)	NO	PRI		

```
2 rows in set (0.02 sec)
```

NB : Nous constatons que au lieu de la clé unique, nous avons plutôt la clé primaire, pourquoi ? Tout simplement parce que tout autre indexes dans une table, doit précéder la clé primaire or c'est n'était notre cas ici.

Supprimant les clés en question :

```
mysql> alter table lo drop unique;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'unique' at line 1
mysql>
```

Ah ok sûrement c'est parce que c'est la clé primaire qui apparaît à la place de la clé unique. Essayant donc avec primaire :

```
mysql>
mysql> alter table lo drop primary key(id);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(id)' at line 1
```

Ça ne marche toujours pas pourquoi ?

Tout simplement parce que nous avons utilisé le style d'efface par contrainte or nous n'avons pas insérer ces clés par contrainte. Faisant donc ce qui suit :


```
mysql> alter table lo drop index id;
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   |     | 0        |       |
| nom   | varchar(30)   | NO   |     |          |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Bravo c'est fait

Comment le faire alors ?

Commençons par ajouter une nouvelle colonne à notre table et insérons la clé primaire.

```
mysql> alter table lo add prenom varchar(30);
Query OK, 0 rows affected (0.32 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> alter table lo add primary key (id);
Query OK, 0 rows affected (0.32 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   | PRI | 0        |       |
| nom   | varchar(30)   | NO   |     |          |       |
| prenom | varchar(30)   | YES  |     | NULL     |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)
```

Ok c'est bon, maintenant ajoutant la clé unique simultanément à nom et prénom.

```
mysql> alter table lo add unique key_v (nom,prenom);
Query OK, 3 rows affected (0.32 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> desc lo;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   | PRI | 0        |       |
| nom   | varchar(30)   | NO   | MUL |          |       |
| prenom | varchar(30)   | YES  |     | NULL     |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

NB : retenons que ce ne sont pas des pratiques encourageante si vous n'aviez pas la parfaite maitrise de la chose.

Retenons aussi les colonnes en multiple (MUL) se comporte comme étant des jumeaux c'est-à-dire qu'elles réagissent à la redondance seulement lors que les valeurs entrées dans ces colonnes ont été répétées simultanément.

Essayez pour en avoir le cœur net.

Alors comment les supprimez sans supprimer les colonnes ?

Très simple, saisissez juste la syntaxe ci-dessous :

```
mysql> alter table lo drop index key_v;  
Query OK, 3 rows affected (0.15 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

VII. CLÉS ÉTRANGÈRES

Les clés étrangères ont pour fonction principale la vérification de l'intégrité de votre base. Elles permettent de s'assurer que vous n'insérez pas de bêtises...

Reprenons l'exemple dans lequel on a une table *Client* et une table *Commande*. Dans la table *Commande*, on a une colonne qui contient une référence au client. Ici, le client numéro 3, M. Nicolas Jacques, a donc passé une commande de trois tubes de colle, tandis que Mme Marie Malherbe (cliente numéro 2) a passé deux commandes, pour du papier et des ciseaux.

Numéro	Nom	Prénom	Email
1	Jean	Dupont	jdupont@email.com
2	Marie	Malherbe	mama@email.com
3	Nicolas	Jacques	Jacques.nicolas@email.com
4	Hadrien	Piroux	happi@email.com



Numéro	Client	Produit	Quantité
1	3	Tube de colle	3
2	2	Rame de papier A4	6
3	2	Ciseaux	2

C'est bien joli, mais que se passe-t-il si **M. Hadrien Piroux** passe une commande de 15 tubes de colle, et qu'à l'insertion dans la table *Commande*, votre doigt dérape et met 45 comme numéro de client ? C'est l'horreur ! Vous avez dans votre base de données une commande passée par un client inexistant, et vous passez votre après-midi du lendemain à vérifier tous vos bons de commande de la veille pour retrouver qui a commandé ces 15 tubes de colle. Magnifique perte de temps !

Ce serait quand même sympathique si, à l'insertion d'une ligne dans la table *Commande*, un gentil petit lutin allait vérifier que le numéro de client indiqué correspond bien à quelque chose dans la table *Client*, non ? Ce lutin, ou plutôt cette lutine, existe ! Elle s'appelle "clé étrangère".

Par conséquent, si vous créez une clé étrangère sur la colonne *client* de la table *Commande*, en lui donnant comme référence la colonne **numéro** de la table *Client*, MySQL ne vous laissera plus jamais insérer un numéro de client inexistant dans la table *Commande*. Il s'agit bien d'une **contrainte** !

Avant d'entamer une danse de joie, parce que quand même le SQL c'est génial, restez concentrés cinq minutes, le temps de lire et retenir quelques points **importants**.

- ❖ Comme pour les index et les clés primaires, il est possible de créer des **clés étrangères composites**. Lorsque vous créez une clé étrangère sur une colonne (ou un groupe de colonnes) – la colonne *client* de *Commande* dans notre exemple –, **un index est automatiquement ajouté** sur celle-ci (ou sur le groupe).
- ❖ Par contre, la colonne (le groupe de colonnes) qui sert de référence - la colonne **numéro** de *Client* - **doit** déjà posséder un index (où être clé primaire bien sûr).
- ❖ La colonne (ou le groupe de colonnes) sur laquelle (lequel) la clé est créée doit être **exactement** du même type que la colonne (le groupe de colonnes) qu'elle (il) référence. Cela implique qu'en cas de clé composite, il faut le même nombre de colonnes dans la clé et la référence. Donc, si *numéro* (dans *Client*) est un **INT UNSIGNED**, *client* (dans *Commande*) doit être de type **INT UNSIGNED** aussi.
- ❖ Tous les moteurs de table ne permettent pas l'utilisation des clés étrangères. Par exemple, **MyISAM** ne le permet pas, contrairement à **InnoDB**.

CRÉATION

Une clé étrangère est un peu plus complexe à créer qu'un index ou une clé primaire, puisqu'il faut deux éléments :

- ❖ La ou les colonnes sur laquelle (lesquelles) on crée la clé - on utilise **FOREIGN KEY** ;
- ❖ La ou les colonnes qui va (vont) servir de référence - on utilise **REFERENCES**.

Lors de la création de la table

Du fait de la présence de deux paramètres, une clé étrangère ne peut que s'ajouter à la suite des colonnes, et pas directement dans la description d'une colonne. Par ailleurs, je vous conseille ici de créer explicitement une contrainte (grâce au mot-clé **CONSTRAINT**) et de lui donner un symbole. En effet, pour les index, on pouvait utiliser leur nom pour les identifier ; pour les clés primaires, le nom de la table suffisait puisqu'il n'y en a qu'une par table. Par contre, pour différencier facilement les clés étrangères d'une table, il est utile de leur donner un nom, à travers la contrainte associée.

À nouveau, je respecte certaines conventions de nommage : mes clés étrangères ont des noms commençant par **fk** (pour **FOREIGN KEY**), suivi du nom de la colonne dans la table puis (si elle s'appelle différemment) du nom de la colonne de référence, le tout séparé par des _ (**fk_client_numero** par exemple).

Donc si on imagine les tables *Client* et *Commande*, pour créer la table *Commande* avec une clé étrangère ayant pour référence la colonne **numéro** de la table *Client*, on utilisera :

Voici la première table :

```
mysql> use loi;
Database changed
mysql>
mysql> create table commande(numero int(3) primary key auto_increment, nom varchar(40), prenom varchar(40));
Query OK, 0 rows affected (0.12 sec)

mysql>
mysql>
```

```
mysql>
mysql> desc commande;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| numero | int(3)        | NO   | PRI | NULL    | auto_increment |
| nom    | varchar(40)   | YES  |     | NULL    |                |
| prenom | varchar(40)   | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.25 sec)
```

DEUXIEME TABLE :

```
mysql> create table achat1(numero int(3) primary key auto_increment, produit varchar(40), prix varchar(40), constraint fk_val foreign key (produit) reference
s command(numero));
Query OK, 0 rows affected (0.33 sec)

mysql>
```

Après la description, on a :

```
mysql>
mysql> desc achat;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| numero | int(3)    | NO   | PRI | NULL    | auto_increment |
| produit | varchar(40) | YES  |     | NULL    |                 |
| prix   | varchar(40) | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

NB : on pouvait aussi le faire après avoir créée la table. Comme ci-dessous :

```
mysql> alter table achat add constraint vk_c foreign key (produit) references commande(
Query OK, 0 rows affected (0.22 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc achat;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| numero | int(3)    | NO   | PRI | NULL    | auto_increment |
| produit | varchar(40) | YES  | MUL | NULL    |                 |
| prix   | varchar(40) | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

VIII. SUPPRESSION D'UNE CLE ETRANGERE

Il peut y avoir plusieurs clés étrangères par table. Par conséquent, lors d'une suppression il faut identifier la clé à détruire. Cela se fait grâce au symbole de la contrainte.

Code : SQL

```
ALTER TABLE nom_table
DROP FOREIGN KEY symbole_contrainte
```

Application :

```
mysql> alter table achat drop foreign key vk_c;
Query OK, 0 rows affected (0.47 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

CREONS MAINTENANT LA TABLE ESPECE POUR QUELQUE EXERCICES.

```
mysql> create table espece(id int(3) primary key auto_increment,nom_courant varchar(33),nom_latin varchar(33) unique,description text) engine=innodb;  
Query OK, 0 rows affected (0.22 sec)
```

```
mysql> desc espece;
```

Field	Type	Null	Key	Default	Extra
id	int(3)	NO	PRI	NULL	auto_increment
nom_courant	varchar(33)	YES		NULL	
nom_latin	varchar(33)	YES	UNI	NULL	
description	text	YES		NULL	

```
4 rows in set (0.02 sec)
```

On met un index **UNIQUE** sur la colonne **nom_latin** pour être sûr que l'on ne rentrera pas deux fois la même espèce. Pourquoi sur le nom latin et pas sur le nom courant ? Tout simplement parce que le nom latin est beaucoup plus rigoureux et réglementé que le nom courant. On peut avoir plusieurs dénominations courantes pour une même espèce ; ce n'est pas le cas avec le nom latin.

Bien, remplissons donc cette table avec les espèces déjà présentes dans la base :

Code : SQL

```
INSERT INTO Espece (nom_courant, nom_latin, description) VALUES  
('Chien', 'Canis canis', 'Bestiole à quatre pattes qui aime les  
caresses et tire souvent la langue'),  
('Chat', 'Felis silvestris', 'Bestiole à quatre pattes qui saute  
très haut et grimpe aux arbres'),  
('Tortue d''Hermann', 'Testudo hermanni', 'Bestiole avec une  
carapace très dure'),  
('Perroquet amazone', 'Alipiopsitta xanthops', 'Joli oiseau  
parleur vert et jaune');
```

```
mysql> insert into espece(nom_courant,nom_latin,description) values ('chien','canis canis','bestiole a quatre pattes qui aime les caresses et tire souvent la langue');
Query OK, 1 row affected (0.00 sec)

mysql> insert into espece(nom_courant,nom_latin,description) values ('chat','felis silvestris','bestiole a quatre pattes qui saute tres haut et grimpe sur les arbres');
Query OK, 1 row affected (0.00 sec)

mysql> insert into espece(nom_courant,nom_latin,description) values ('tortue d\hermann','testudo hermanni','bestiole avec une carapace tres dure');
Query OK, 1 row affected (0.00 sec)

mysql> insert into espece(nom_courant,nom_latin,description) values ('perroquet amazone','alipiopsitta xanthops','joli oiseau parleur vert et jaune');
Query OK, 1 row affected (0.00 sec)
```

Ce qui donnera la table ci-dessous :

id	nom_courant	nom_latin	description
1	Chien	Canis canis	Bestiole à quatre pattes qui aime les caresses et tire souvent la langue
2	Chat	Felis silvestris	Bestiole à quatre pattes qui saute très haut et grimpe aux arbres
3	Tortue d'Hermann	Testudo hermanni	Bestiole avec une carapace très dure
4	Perroquet amazone	Alipiopsitta xanthops	Joli oiseau parleur vert et jaune

Soit :

```
mysql> select * from espece;
+----+-----+-----+-----+
| id | nom_courant | nom_latin | description |
+----+-----+-----+-----+
| 1 | chien | canis canis | bestiole a quatre pattes qui aime les caresses et tire souvent la langue |
| 2 | chat | felis silvestris | bestiole a quatre pattes qui saute tres haut et grimpe sur les arbres |
| 3 | tortue d'hermann | testudo hermanni | bestiole avec une carapace tres dure |
| 4 | perroquet amazone | alipiopsitta xanthops | joli oiseau parleur vert et jaune |
+----+-----+-----+-----+
4 rows in set (0.00 sec)
```

J'ai donc précisé un peu (si je donne juste l'ordre, c'est comme si je mettais "carnivore" au lieu de "chat" - ou "chien" d'ailleurs).

Bien, mais cela ne suffit pas ! Il faut également modifier notre table ***Animal***. Nous allons ajouter une colonne ***espece_id*** , qui contiendra l'***id*** de l'espèce à laquelle appartient l'animal, et **remplir cette colonne**. Ensuite nous pourrons supprimer la colonne ***espece***, qui n'aura plus de raison d'être.

La colonne **espece_id** sera une clé étrangère ayant pour référence la colonne *id* de la table **Espece**. Je rappelle donc que ça signifie qu'il ne sera pas possible d'insérer dans *espece_id* un nombre qui n'existe pas dans la colonne *id* de la table **Espece**.

LA TABLE ANIMALE

```
mysql> create table animal(id int(3) primary key auto_increment, espece varchar(40), sexe char(40), ddn year, nom varchar(55), commentaire text) engine=innodb;
Query OK, 0 rows affected (0.36 sec)

mysql> desc animal;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| id         | int(3)    | NO   | PRI | NULL    | auto_increment |
| espece     | varchar(40) | YES  |     | NULL    |              |
| sexe       | char(40)   | YES  |     | NULL    |              |
| ddn        | year(4)    | YES  |     | NULL    |              |
| nom        | varchar(55) | YES  |     | NULL    |              |
| commentaire | text      | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Nous allons ajouter une colonne **espece_id** à notre table :

```
mysql> alter table animal add espece_id int(4);
Query OK, 0 rows affected (0.33 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc animal;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| id         | int(3)    | NO   | PRI | NULL    | auto_increment |
| espece     | varchar(40) | YES  |     | NULL    |              |
| sexe       | char(40)   | YES  |     | NULL    |              |
| ddn        | year(4)    | YES  |     | NULL    |              |
| nom        | varchar(55) | YES  |     | NULL    |              |
| commentaire | text      | YES  |     | NULL    |              |
| espece_id  | int(4)     | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.04 sec)
```

Opération de remplissage :

```
mysql> insert into animal (espece, espece_id) values ('chien',1), ('chat',2), ('tortue',3), ('perroquet',4);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0
```


D'où :

```
mysql> select * from animal;
+----+-----+-----+-----+-----+-----+-----+
| id | espece | sexe | ddn | nom | commentaire | espece_id |
+----+-----+-----+-----+-----+-----+-----+
| 1 | chien | NULL | NULL | NULL | NULL | 1 |
| 2 | chat | NULL | NULL | NULL | NULL | 2 |
| 3 | tortue | NULL | NULL | NULL | NULL | 3 |
| 4 | perroquet | NULL | NULL | NULL | NULL | 4 |
+----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

SUPPRESSION DE LA COLONNE ESPECE :

```
mysql> alter table animal drop column espece;
Query OK, 4 rows affected (0.33 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Nouvelle introduction, **colum**.

AJOUTER LA CLE ETRANGERE

```
mysql> alter table animal add constraint fk_espece_id foreign key (espece_id) references espece(id);
Query OK, 4 rows affected (0.33 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Par description :

```
mysql> desc animal;
+----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| id | int(3) | NO | PRI | NULL | auto_increment |
| sexe | char(40) | YES | | NULL | |
| ddn | year(4) | YES | | NULL | |
| nom | varchar(55) | YES | | NULL | |
| commentaire | text | YES | | NULL | |
| espece_id | int(4) | YES | MUL | NULL | |
+----+-----+-----+-----+-----+-----+
6 rows in set (0.02 sec)
```

Pour tester l'efficacité de la **clé étrangère**, essayons d'ajouter un animal dont l'**espece_id** est 5 (qui n'existe donc pas dans la table **Espece**) :

```
mysql> insert into animal (nom,espece_id,ddn) values ('Caouette',5,'2009-02-15');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('loi`.`animal`, CONSTRAINT `fk_espece_id` FOREIGN KEY (`espece_id`) REFERENCES `espece` (`id`))
mysql>
```

```
mysql> select * from animal;
```

id	sexe	ddn	nom	commentaire	espece_id
1	NULL	NULL	NULL	NULL	1
2	NULL	NULL	NULL	NULL	2
3	NULL	NULL	NULL	NULL	3
4	NULL	NULL	NULL	NULL	4

```
4 rows in set (0.00 sec)
```

Elle n'est pas belle la vie ?

J'en profite également pour ajouter une contrainte **NOT NULL** sur la colonne **espece_id**. Après tout, si **espece** ne pouvait pas être **NULL**, pas de raison qu'**espece_id** puisse l'être ! Ajoutons également l'index **UNIQUE** sur (**nom, espece_id**) dont on a déjà discuté.

```
mysql> alter table animal modify espece_id int(4) not null;
Query OK, 4 rows affected (0.53 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> desc animal;
```

Field	Type	Null	Key	Default	Extra
id	int(3)	NO	PRI	NULL	auto_increment
sexe	char(40)	YES		NULL	
ddn	year(4)	YES		NULL	
nom	varchar(55)	YES		NULL	
commentaire	text	YES		NULL	
espece_id	int(4)	NO	MUL	NULL	

```
6 rows in set (0.00 sec)
```

AJOUTANT MAINTENANT SIMULTANEMENT LES INDEXES UNIQUES

```
mysql> create unique index id_index on animal (nom,espece_id);
Query OK, 4 rows affected (0.47 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> desc animal;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| id         | int(3)    | NO   | PRI | NULL    | auto_increment |
| sexe      | char(40)  | YES  |     | NULL    |              |
| ddn       | year(4)   | YES  |     | NULL    |              |
| nom       | varchar(55)| YES  | MUL | NULL    |              |
| commentaire | text      | YES  |     | NULL    |              |
| espece_id | int(4)    | NO   | MUL | NULL    |              |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

Notez qu'il n'était pas possible de mettre la contrainte **NOT NULL** à la création de la colonne, puisque tant que l'on n'avait pas rempli **espece_id**, elle contenait **NULL** pour toutes les lignes.

Voilà, nos deux tables sont maintenant prêtes. Mais avant de vous lâcher dans l'univers merveilleux des jointures et des sous requêtes, nous allons encore compliquer un peu les choses. Parce que c'est bien joli pour un éleveur de pouvoir reconnaître un chien d'un chat, mais il serait de bon ton de reconnaître également un berger allemand d'un teckel, non ?

Par conséquent, nous allons encore ajouter deux choses à notre base. D'une part, nous allons ajouter une table *Race*, basée sur le même schéma que la table **Espece**. Il faudra également ajouter une colonne à la table *Animal*, qui contiendra l'*id* de la race de l'animal. Contrairement à la colonne **espece_id**, celle-ci pourra être **NULL**. Il n'est pas impossible que nous accueillions des bâtards, ou que certaines espèces que nous élevons ne soient pas classées en plusieurs races différentes. Ensuite, nous allons garder une trace du pedigree de nos animaux. Pour ce faire, il faut pouvoir connaître ses parents. Donc, nous ajouterons deux colonnes à la table *Animal* : **pere_id** et **mere_id**, qui contiendront respectivement l'*id* du père et l'*id* de la mère de l'animal.

Ce sont toutes des commandes que vous connaissez, donc je ne détaille pas plus.

CREATION DE LA TABLE RACE

```
mysql> create table race (id int(5) primary key auto_increment,nom varchar(40) not null, espece_id int(4) not null,description text,constraint fk_race_espece_id foreign key (espece_id) references espece(id)) engine=innodb;
Query OK, 0 rows affected (0.32 sec)
```

Après description :

```
mysql> desc race;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	NO	PRI	NULL	auto_increment
nom	varchar(40)	NO		NULL	
espece_id	int(4)	NO	MUL	NULL	
description	text	YES		NULL	

4 rows in set (0.00 sec)

REEMPLISSAGE DE LA TABLE

```
mysql> insert into race (nom,espece_id,description) values ('Berger allemand',1,'chien sportif et elegant au pelage dense, noire-marron-fauve, noir ou gris.');
```

Query OK, 1 row affected (0.06 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Berger blanc suisse',1,'petit chien au corps compact, avec des pattes courtes mais bien proportionnées et au pelage tricolore ou bicolore');
```

Query OK, 1 row affected (0.08 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Boxer',1,'chien de taille moyenne, au poil ras et de couleur fauve ou bringé avec quelques marques blanches');
```

Query OK, 1 row affected (0.30 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Bleu russe',2,'chat aux yeux verts et a la robe epaisse et argentée');
```

Query OK, 1 row affected (0.30 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Maine coon',2,'chat de grande taille, a poils mi-long.');
```

Query OK, 1 row affected (0.32 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Singapura',2,'Chat de petite taille aux grands yeux en amandes.');
```

Query OK, 1 row affected (0.10 sec)

```
mysql> insert into race (nom,espece_id,description) values ('Sphynx',2,'chat sans poils');
```

Query OK, 1 row affected (0.08 sec)

Le tout donnera :

```
mysql> select * from race;
```

id	nom	espece_id	description
1	Berger allemand	1	chien sportif et elegant au pelage dense, noire-marron-fauve, noir ou gris.
2	Berger blanc suisse	1	petit chien au corps compact, avec des pattes courtes mais bien proportionnées et au pelage tricolore ou bicolore
3	Boxer	1	chien de taille moyenne, au poil ras et de couleur fauve ou bringé avec quelques marques blanches
4	Bleu russe	2	chat aux yeux verts et a la robe epaisse et argentée
5	Maine coon	2	chat de grande taille, a poils mi-long.
6	Singapura	2	Chat de petite taille aux grands yeux en amandes.
7	Sphynx	2	chat sans poils

7 rows in set (0.25 sec)

AJOUT DE LA COLONNE RACE_ID A LA TABLE ANIMALE

```
mysql> alter table animal add column race_id int(4);
Query OK, 4 rows affected (0.24 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> desc animal;
```

Field	Type	Null	Key	Default	Extra
id	int(3)	NO	PRI	NULL	auto_increment
sexe	char(40)	YES		NULL	
ddn	year(4)	YES		NULL	
nom	varchar(55)	YES	MUL	NULL	
commentaire	text	YES		NULL	
espece_id	int(4)	NO	MUL	NULL	
race_id	int(4)	YES		NULL	

7 rows in set (0.00 sec)

AJOUTONS UNE CLE ETRANGERE A LA TABLE ANIMAL

```
mysql> alter table animal add constraint fk_race_id foreign key (race_id) references race(id);
Query OK, 4 rows affected (0.47 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> desc animal;
```

Field	Type	Null	Key	Default	Extra
id	int(3)	NO	PRI	NULL	auto_increment
sexe	char(40)	YES		NULL	
ddn	year(4)	YES		NULL	
nom	varchar(55)	YES	MUL	NULL	
commentaire	text	YES		NULL	
espece_id	int(4)	NO	MUL	NULL	
race_id	int(4)	YES	MUL	NULL	

7 rows in set (0.00 sec)

REMPLISSAGE DE LA COLONNE

```
mysql> update animal set race_id=1 where id in (1,13,20,18,22,25,26,28);
Query OK, 1 row affected (0.39 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update animal set race_id=2 where id in (12,14,19,7);
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> update animal set race_id=3 where id in (23,17,21,27);
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> update animal set race_id=4 where id in (33,35,37,41,44,31,3);
Query OK, 1 row affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update animal set race_id=5 where id in (43,40,30,32,42,34,39,8);
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> update animal set race_id=6 where id in (29,36,38);
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

Cela donnera donc:

```
mysql> select * from animal;
+----+-----+-----+-----+-----+-----+-----+
| id | sexe | ddn  | nom  | commentaire | espece_id | race_id |
+----+-----+-----+-----+-----+-----+-----+
| 1  | NULL | NULL | NULL | NULL        | 1         | 1       |
| 2  | NULL | NULL | NULL | NULL        | 2         | NULL    |
| 3  | NULL | NULL | NULL | NULL        | 3         | 4       |
| 4  | NULL | NULL | NULL | NULL        | 4         | NULL    |
+----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

AJOUT DES COLONNES MERE_ID ET PERE_ID DANS LA TABLE ANIMAL

```
mysql> alter table animal add mere_id int(4) not null;
Query OK, 4 rows affected (0.25 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> alter table animal add pere_id int(4) not null;
Query OK, 4 rows affected (0.24 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> desc animal;
```

Field	Type	Null	Key	Default	Extra
id	int(3)	NO	PRI	NULL	auto_increment
sexe	char(40)	YES		NULL	
ddn	year(4)	YES		NULL	
nom	varchar(55)	YES	MUL	NULL	
commentaire	text	YES		NULL	
espece_id	int(4)	NO	MUL	NULL	
race_id	int(4)	YES	MUL	NULL	
mere_id	int(4)	NO		NULL	
pere_id	int(4)	NO		NULL	

```
9 rows in set (0.01 sec)
```

Référencier une clé étrangère de la même table n'est pas prête de marcher comme c'est fait ci-dessous :

```
mysql> alter table animal add constraint fk_mere_id foreign key (mere_id) references animal(id);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('loi'.<result 2 when explaining filename '#sql-12bc_1d', CONSTRAINT `fk_mere_id` FOREIGN KEY (`mere_id`) REFERENCES `animal` (`id`))
mysql>
```

REPLISSAGE DES COLONNES MERE_ID ET PERE_ID

```
mysql> update animal set mere_id=18, pere_id=22 where id=1;
Query OK, 1 row affected (0.32 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> update animal set mere_id=7, pere_id=21 where id=10;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0
```

```
mysql> update animal set mere_id=41, pere_id=31 where id=3;
Query OK, 1 row affected (0.33 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> update animal set mere_id=40, pere_id=30 where id=2;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Du coup on a :

```
mysql> select * from animal;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | sexe | ddn  | nom  | commentaire | espece_id | race_id | mere_id | pere_id |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | NULL | NULL | NULL | NULL        | 1         | 1       | 18      | 22      |
| 2  | NULL | NULL | NULL | NULL        | 2         | NULL    | 40      | 30      |
| 3  | NULL | NULL | NULL | NULL        | 3         | 4       | 41      | 31      |
| 4  | NULL | NULL | NULL | NULL        | 4         | NULL    | 0       | 0       |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

EN RÉSUMÉ

- Une clé primaire permet d'identifier chaque ligne de la table de manière unique : c'est à la fois une contrainte d'unicité et une contrainte **NOT NULL**.
- Chaque table **doit définir une clé primaire**, et une table ne peut avoir qu'**une seule** clé primaire.
- Une clé étrangère permet de définir une relation entre deux tables, et d'assurer la cohérence des données.

FIN !